

# ELEC 475 Lab 3

Names: Sage Franco, Thomas Wilkinson

Student numbers: 20280229, 20269155

Date: November 11<sup>th</sup>, 2025.

## What is your network architecture?

You should describe the structure in sufficient detail so that it can be reimplemented solely from the textual description. You should also have proper citations to the techniques that you used.

The model used in this lab is an encoder/decoder segmentation network designed for pixel-wise semantic segmentation. The inputs, which consisted of 3 channel (RGB) 512x512 images were fed into the encoder. The encoder used a convolutional feature extractor backbone named MobileNetV3-Small [1] that was pre-trained on the ImageNet dataset. Feature maps were extracted from three intermediate layers. The first, the low-level, was extracted after the second layer with channel dimensions of 24. The second, the mid-level, was extracted after the fifth layer with channel dimensions of 40. The third, the high-level, was extracted after the 12th layer with channel dimensions of 576. Following the encoder is the bottleneck. For this stage, the Atrous Spatial Pyramid Pooling (ASPP) stage was used to capture multi-scale contextual information from the encoder's high level feature map [2]. Four parallel convolutional branches with atrous convolutions used dilation rates of 1, 6, 12, and 18. When dilation was equal to 1, a 1x1 convolution was applied. When it was larger, a 3x3 convolution followed by a batch normalization and a SiLU activation function was applied, followed by another 1x1 convolution, batch normalization, and SiLU. Following this was a pooling stage consisting of an adaptive average pool, a 1x1 convolution, a batch normalization, and a SiLU activation function. Finally, the outputs of all branches were concatenated and passed through another 1x1 convolution, batch normalization, and SiLU activation to produce a 256-channel fused representation. The decoder follows the general structure of a U-Net [3]. The ASPP output (256 channel tensor) was up sampled to the resolution of the mid-level features (40) and concatenated along the channel dimension. A 3x3 convolution, batch normalization, and SiLU activation function was then applied to the concatenated output. The resulting feature map was up sampled again to match the spatial size of the low-level features (24) and concatenated with them. Another 3x3 convolution, batch normalization, SiLU was applied. Finally, the output was up sampled to the original input resolution (512x512) and passed through a 1x1 convolution to predict per-pixel class logits.

## Describe all hyperparameters used in training. How long did the training take? Include all loss plots.

Training was conducted for 50 epochs using a batch size of 4, with only the training data shuffled at each epoch. The network was optimized using the AdamW optimizer with a learning rate of  $3e-4$  and a weight decay of  $1e-4$  to prevent overfitting. A cosine annealing warm restart learning rate scheduler ( $T_0=10$ ,  $T_{mult}=2$ ) was used to cyclically reduce the learning rate and promote more stable convergence. The overall loss function combined a weighted cross-entropy loss, where the background class was downweighed to 0.2 to address class imbalance with a Dice loss to enhance overlap accuracy between predicted and ground-truth masks. Data augmentation during training included random horizontal flipping, color jittering, and random resized cropping, while validation images were only resized and normalized. All convolutional layers used the Kaiming (He) weights

initialization inherited from the pretrained MobileNetV3-Small backbone, ensuring stable gradient flow and effective feature reuse. Training took 30 to 60 minutes on average. The loss plots for the custom model and knowledge distillation techniques can be found in Figures 1, 2, and 3 below.

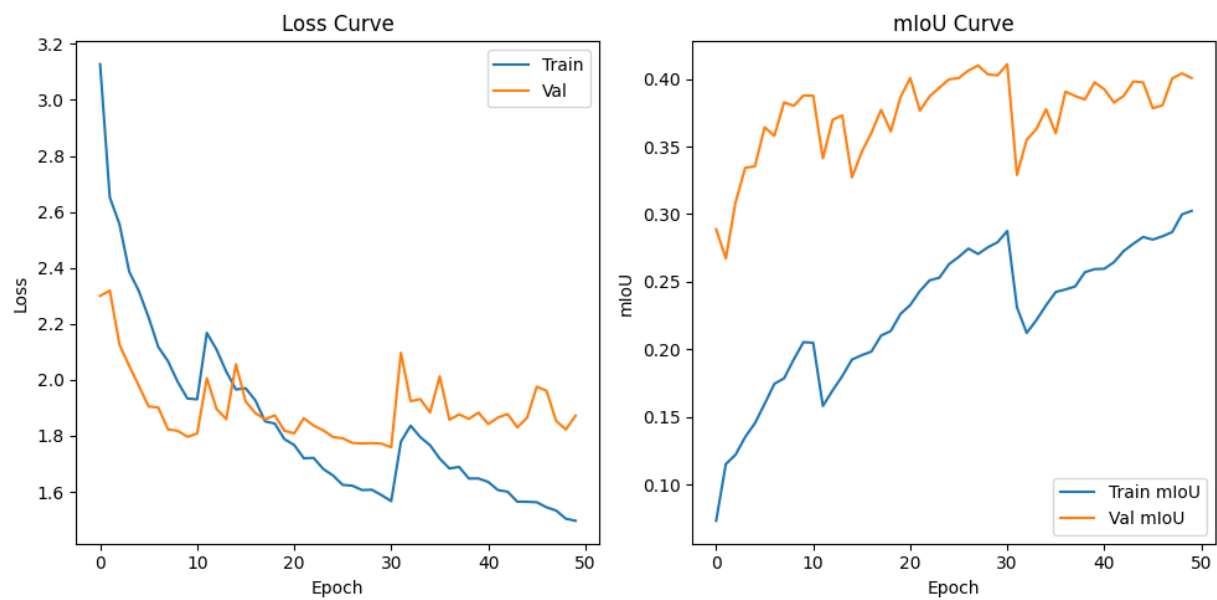


Figure 1: Loss and mIoU plots for custom model.

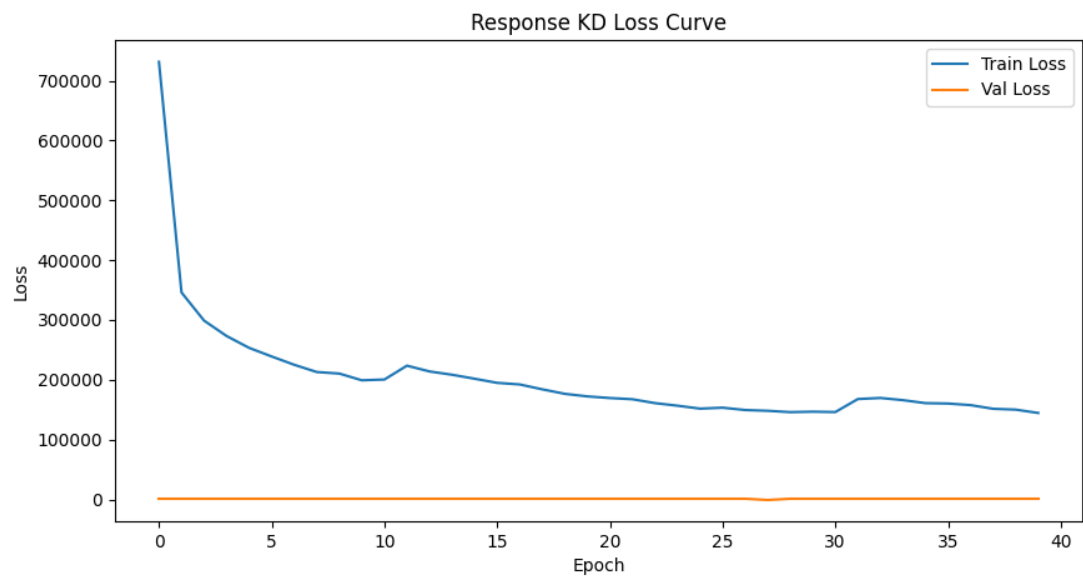


Figure 2: Training and validation loss for response-based knowledge distillation.

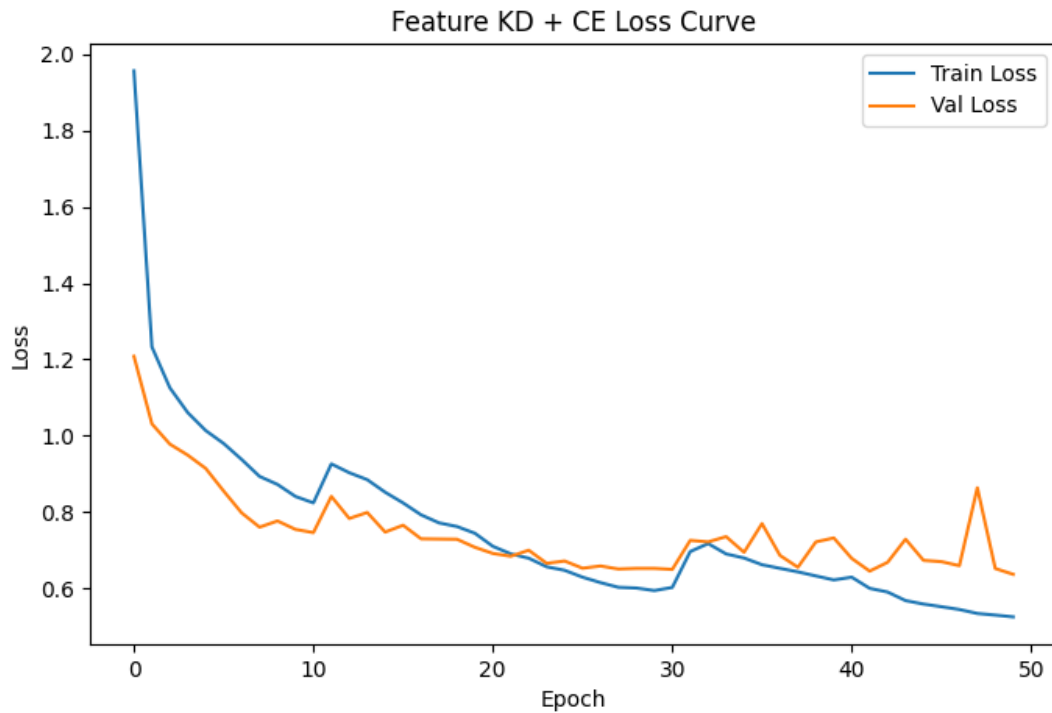


Figure 3: Training and validation loss for feature-based knowledge distillation.

## Describe your experiments.

The group used the NVIDIA A100 Tensor Core GPU on Google Colab to train and test the models. Model training took 30 – 60 minutes on average, while testing took around 1 minute. Over 10 versions of the model were developed, each improving model architecture or training hyperparameters. The best performing version had a mean mIoU of 0.367 and a maximum mIoU of 0.41 during training.

Three knowledge distillation strategies were compared: baseline (no distillation), feature-based knowledge distillation, and response-based knowledge distillation. Feature-based knowledge distillation slightly reduced accuracy, while response-based significantly improved it.

| Knowledge Distillation | mIoU   | # of Parameters | Inference Speed (ms per image) |
|------------------------|--------|-----------------|--------------------------------|
| Baseline (without)     | 0.367  | 2.5M            | 337.47                         |
| Feature based          | 0.361  | 2.5M            | 195.25                         |
| Response based         | 0.4925 | 2.5M            | 230.71                         |

The initial version of the model used cross entropy loss without weightings which caused the model to predict all pixels as background. After introducing both weighted cross entropy loss and DICE loss, as well as skip connections, the model started to predict object regions, though the

predictions were splotchy and not very accurate. However, the predictions were splotchy and inaccurate. The final version switched to a U-net style architecture with more skip connections, which produced noticeably more accurate and coherent segmentations. Applying response-based distillation on this final version improved the predicted object boundaries and reduced the amount of background pixels being predicted as objects.

#### Failure cases from the model:

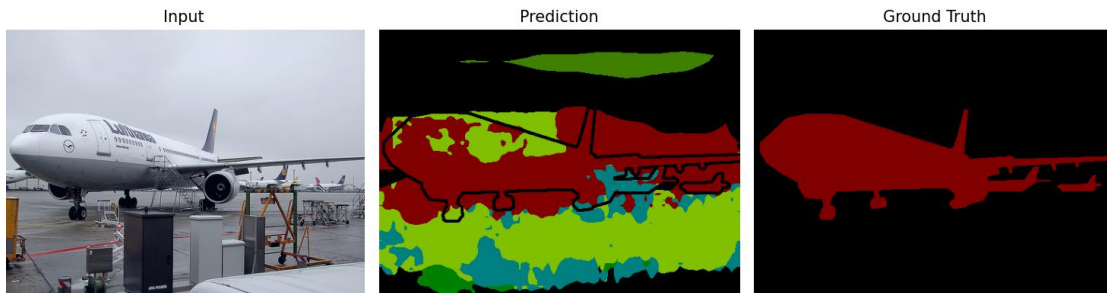


Figure 4: Failure case 1.

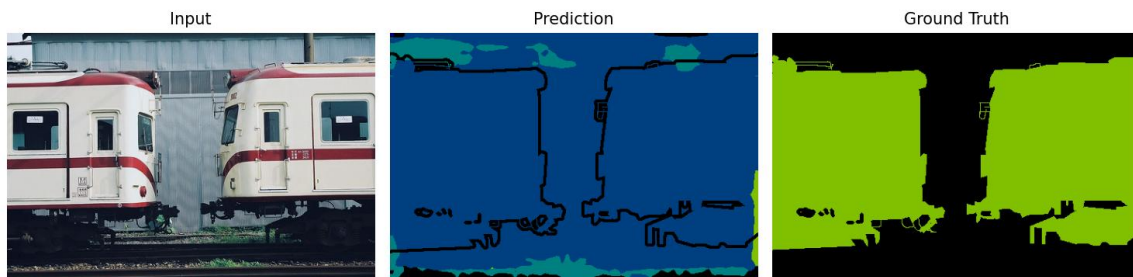


Figure 5: Failure case 2.

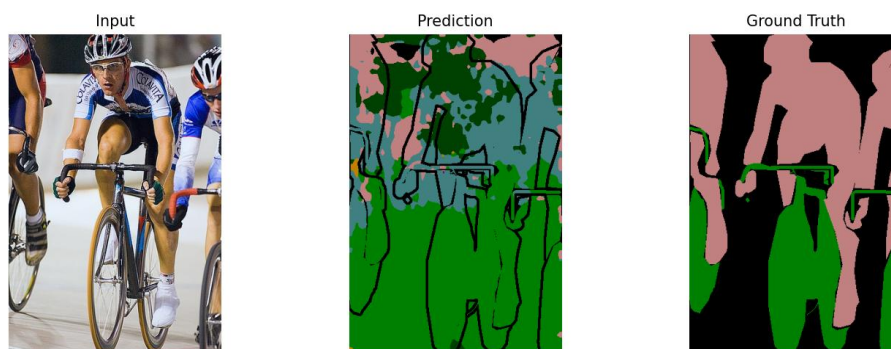
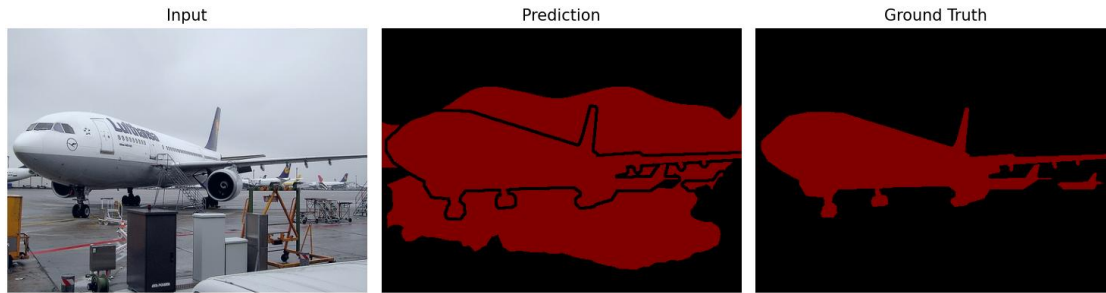
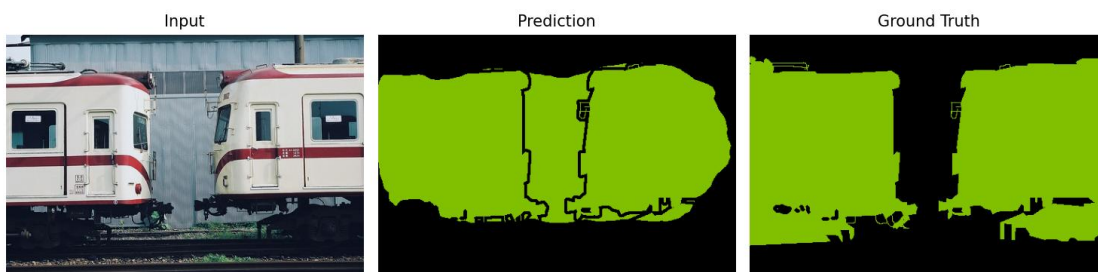


Figure 6: Failure case 3.

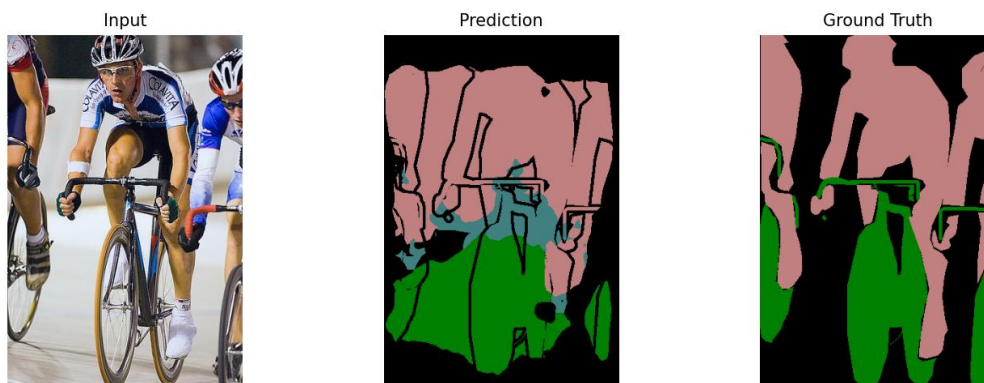
**Final version of custom model without any knowledge distillation:**



*Figure 7: Final version with no knowledge distillation example 1.*



*Figure 8: Final version with no knowledge distillation example 2.*



*Figure 9: Final version with no knowledge distillation example 3.*

### With Response-based Knowledge Distillation:

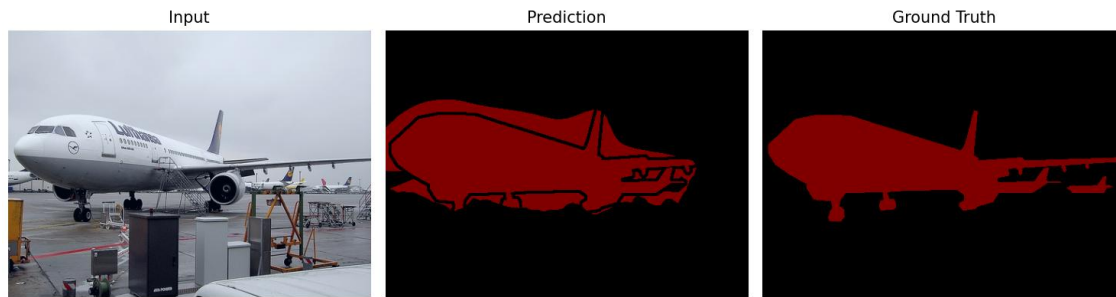


Figure 10: Final version with knowledge distillation example 1.

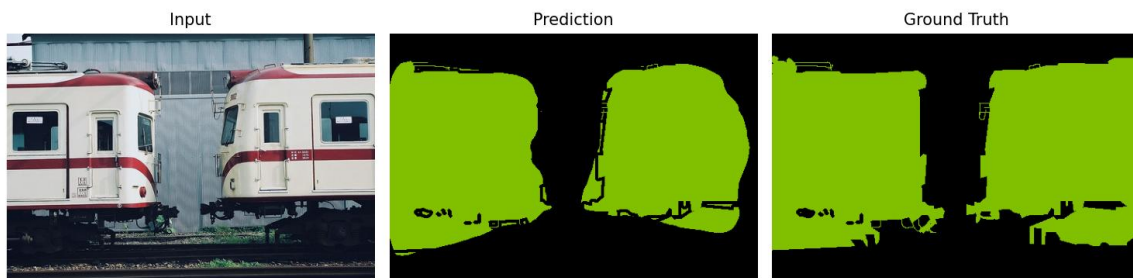


Figure 11: Final version with knowledge distillation example 2.

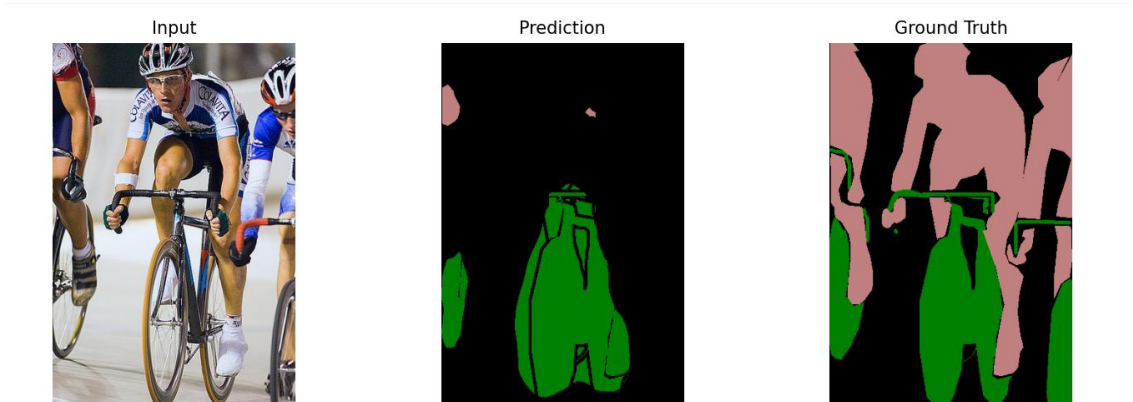


Figure 12: Final version with knowledge distillation example 3.

## Discuss the performance of your system.

The final custom model worked relatively well. The predicted classes were usually accurate, while the object predictions didn't always stay in the expected boundary. There were usually more false positives, meaning background pixels surrounding the object were being predicted as the object. The response-based knowledge distillation was very beneficial, increasing the mIoU by 0.13, and reduced most of the false positives.

The predictions were very messy and inaccurate in the beginning, and it was challenging getting the model to improve. I used an LLM to help recommend new changes and rewrite the code, but it got stuck in a loop of recommending changes and promising high accuracy, and the new models were not resulting in better predictions or a higher mIoU. To overcome this, I got ideas to improve the model from looking at the course content and started a fresh chat with a new prompt.

## References

- [1] A. Howard *et al.*, “Searching for MobileNetV3,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea (South): IEEE, Oct. 2019, pp. 1314–1324. doi: 10.1109/ICCV.2019.00140.
- [2] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” in *Computer Vision – ECCV 2018*, vol. 11211, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., in *Lecture Notes in Computer Science*, vol. 11211. , Cham: Springer International Publishing, 2018, pp. 833–851. doi: 10.1007/978-3-030-01234-2\_49.
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” May 18, 2015, *arXiv*: arXiv:1505.04597. doi: 10.48550/arXiv.1505.04597.